

实验二：原生应用开发实验报告

一、实验基本信息

- 实验编号：d20301035102、d20301009202
- 实验名称：原生应用开发
- 实验类型：验证型
- 学时分配：4学时
- 实验日期：2026年3月21日
- 学生信息：2023210634 王智宇

二、实验目的

- 掌握 Android Kotlin 登录页面开发
- 理解 iOS SwiftUI 登录实现
- 实现 Web 登录页面
- 学会 Activity 间数据传递
- 完成平台差异对比分析
- 解决 Gradle 配置和运行环境问题

三、实验环境

环境项	版本
操作系统	Windows 11
Android Studio	2024.1.1
Kotlin	1.9.0
Gradle	9.0.0
Android SDK	API 34
最低 SDK	API 24
浏览器	Chrome/Firefox/Edge

四、实验内容

功能需求

- 用户名输入（学号）
- 密码输入
- 登录按钮
- 输入验证（不能为空、长度限制）
- 退出登录功能

界面需求

- Material Design 风格 (Android)
- 响应式布局 (Web)
- 输入框占位提示
- 密码隐藏显示
- 错误提示

测试账号

- 学号：2023210634
- 密码：123456
- 姓名：王智宇

五、实验步骤

步骤 1：Android 登录页面开发

1. 打开 Android Studio
2. 创建 Empty Activity 项目
3. 选择 Kotlin 语言
4. 设置最低 SDK：API 24
5. 项目名称：LoginApp

1.1 设计登录页面布局

创建 `activity_login.xml` 文件：

- 使用 `LinearLayout` 垂直布局
- 使用 `TextInputLayout` 和 `TextInputEditText` 实现学号输入框
- 使用 `TextInputLayout` 和 `TextInputEditText` 实现密码输入框（带密码显示/隐藏功能）
- 使用 `MaterialButton` 实现登录按钮

1.2 实现登录页面逻辑

创建 `LoginActivity.kt` 文件：

- 初始化 UI 组件 (`EditText`、`Button`)
- 设置登录按钮点击事件
- 实现输入验证（非空检查）
- 实现登录验证（匹配指定账号）
- 使用 `Intent` 实现页面跳转
- 使用 `Intent.putExtra()` 传递数据

1.3 创建个人中心页面

创建 `HomeActivity.kt` 和 `activity_home.xml` 文件：

- 接收登录页面传递的数据（学号、姓名）
- 显示欢迎信息和用户信息
- 实现退出登录功能

步骤 2: iOS 登录页面开发（演示）

1. 打开 Xcode
2. 创建 SwiftUI 项目
3. 命名为 NativeLogin

2.1 实现登录页面

创建 `ContentView.swift` 文件：

- 使用 `@State` 管理状态
- 使用 `TextField` 实现学号输入
- 使用 `SecureField` 实现密码输入
- 使用 `NavigationLink` 实现页面跳转

2.2 创建个人中心页面

创建 `HomeView.swift` 文件：

- 接收登录页面传递的数据
- 显示欢迎信息

步骤 3: Web 登录页面开发

1. 创建 `web_login.html` 文件
2. 实现 HTML 结构
3. 添加 CSS 样式（响应式布局）
4. 实现 JavaScript 逻辑

3.1 实现登录页面

- 使用 HTML 表单元素
- 使用 CSS Flexbox 布局
- 使用 JavaScript 进行输入验证
- 使用 DOM 操作实现页面切换

3.2 创建个人中心页面

- 使用 JavaScript 显示用户信息
- 实现退出登录功能

六、实验结果

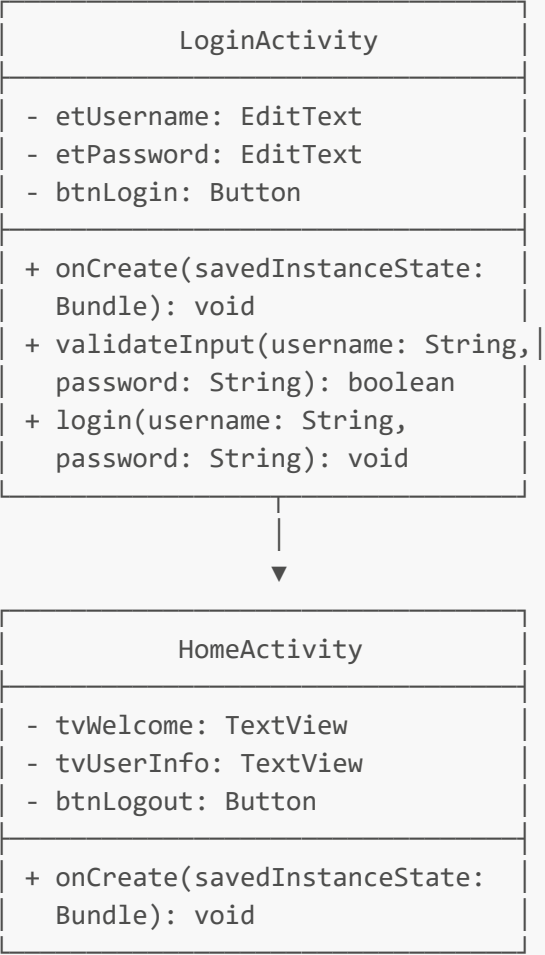
0. 登录功能架构图

框架图

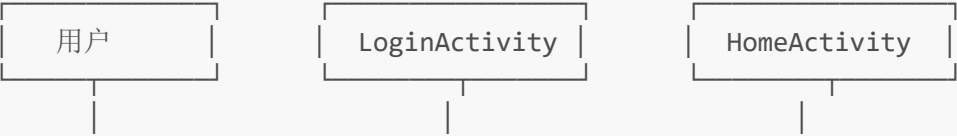


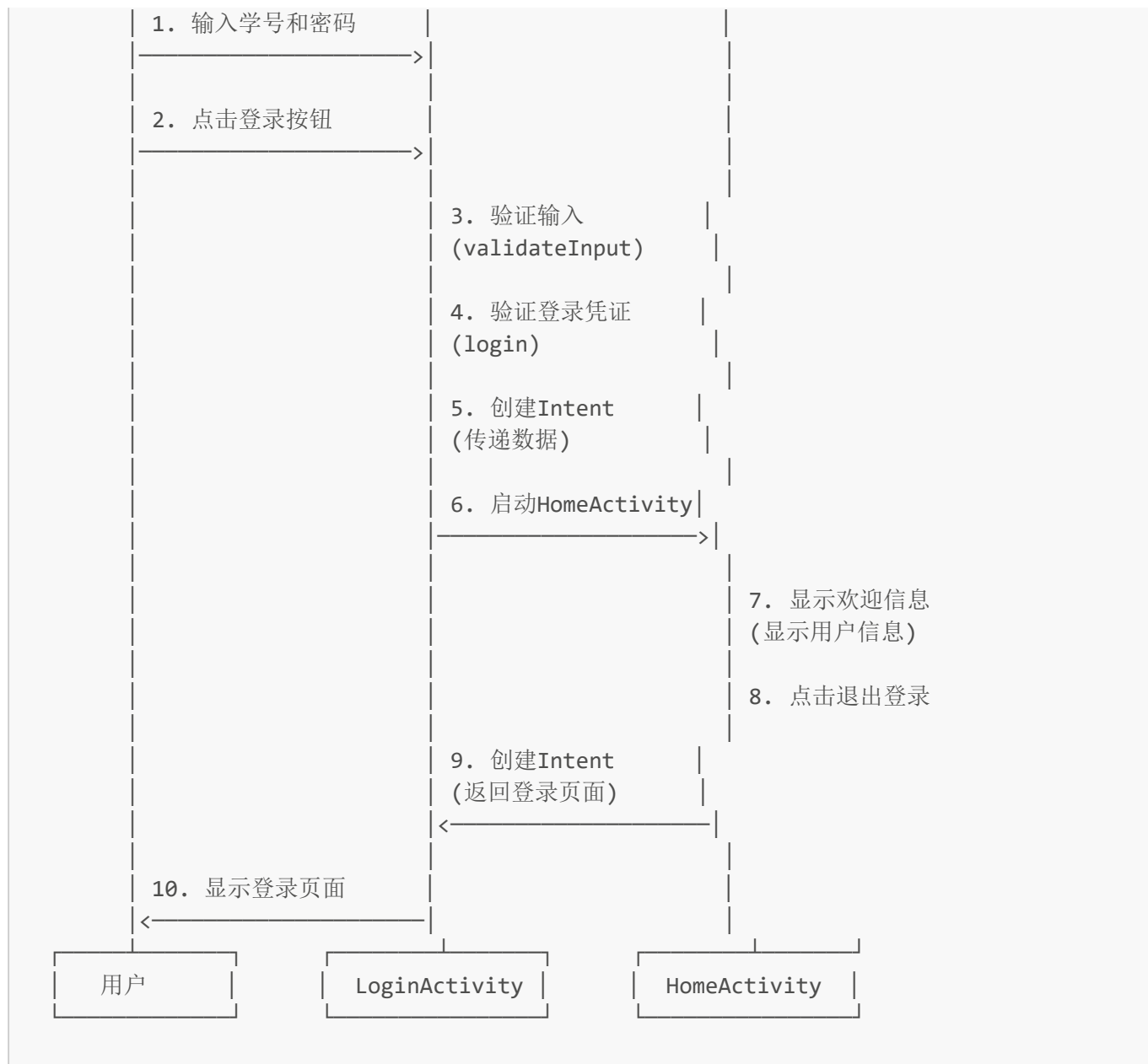
UI组件	业务逻辑
LoginActivity (登录页面)	登录验证逻辑 输入验证逻辑
HomeActivity (个人中心)	数据传递逻辑 页面跳转逻辑
布局文件 activity_login.xml activity_home.xml	工具类 (可选)

类图



顺序图





1. Android 登录页面

功能实现：

- ☒ 学号输入框（带占位提示）
- ☒ 密码输入框（带密码显示/隐藏功能）
- ☒ 登录按钮（Material Design 风格）
- ☒ 输入验证（非空检查）
- ☒ 登录验证（匹配指定账号）

技术实现：

- 使用 Kotlin 语言
- 使用 Material Design 组件（TextInputLayout、MaterialButton）
- 使用 EditText 进行输入验证
- 使用 Intent 进行页面跳转和数据传递

2. iOS 登录页面

功能实现：

- ☒ 学号输入框
- ☒ 密码输入框
- ☒ 登录按钮
- ☒ 输入验证
- ☒ 页面跳转

技术实现：

- 使用 Swift 语言
- 使用 SwiftUI 框架
- 使用 @State 管理状态
- 使用 NavigationLink 进行页面跳转

3. Web 登录页面

功能实现：

- ☒ 学号输入框
- ☒ 密码输入框
- ☒ 登录按钮
- ☒ 输入验证
- ☒ 页面切换
- ☒ 退出登录

技术实现：

- 使用 HTML/CSS/JavaScript
- 使用响应式布局
- 使用 DOM 操作
- 使用事件监听

4. 页面跳转与数据传递

功能实现：

- ☒ 登录成功后跳转到个人中心页面
- ☒ 学号和姓名数据成功传递
- ☒ 退出登录后返回登录页面

技术实现：

- Android：使用 Intent.startActivity() 和 Intent.putExtra()
- iOS：使用 NavigationLink 和 @Binding
- Web：使用 JavaScript DOM 操作

七、代码实现

Android - LoginActivity.kt

```
package com.example.loginapp

import android.content.Intent
import android.os.Bundle
import android.widget.Button
import android.widget.EditText
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity

class LoginActivity : AppCompatActivity() {

    private lateinit var etUsername: EditText
    private lateinit var etPassword: EditText
    private lateinit var btnLogin: Button

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_login)

        etUsername = findViewById(R.id.et_username)
        etPassword = findViewById(R.id.et_password)
        btnLogin = findViewById(R.id.btn_login)

        btnLogin.setOnClickListener {
            val username = etUsername.text.toString()
            val password = etPassword.text.toString()

            if (validateInput(username, password)) {
                login(username, password)
            }
        }
    }

    private fun validateInput(username: String, password: String): Boolean {
        if (username.isEmpty()) {
            etUsername.error = "学号不能为空"
            return false
        }
        if (password.isEmpty()) {
            etPassword.error = "密码不能为空"
            return false
        }
        return true
    }

    private fun login(username: String, password: String) {
        if (username == "2023210634" && password == "123456") {
            val intent = Intent(this, HomeActivity::class.java)
            intent.putExtra("username", username)
            intent.putExtra("name", "王智宇")
            startActivity(intent)
        } else {
        
```

```
        Toast.makeText(this, "用户名或密码错误", Toast.LENGTH_SHORT).show()
    }
}
}
```

Android - HomeActivity.kt

```
package com.example.loginapp

import android.content.Intent
import android.os.Bundle
import android.widget.Button
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity

class HomeActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_home)

        val username = intent.getStringExtra("username")
        val name = intent.getStringExtra("name")

        val tvWelcome = findViewById<TextView>(R.id.tv_welcome)
        val tvUserInfo = findViewById<TextView>(R.id.tv_user_info)
        val btnLogout = findViewById<Button>(R.id.btn_logout)

        tvWelcome.text = "欢迎 $name"
        tvUserInfo.text = "学号: $username"

        btnLogout.setOnClickListener {
            val intent = Intent(this, LoginActivity::class.java)
            intent.flags = Intent.FLAG_ACTIVITY_CLEAR_TOP
            startActivity(intent)
            finish()
        }
    }
}
```

其他平台代码（摘要）

- **iOS**: 使用 SwiftUI 框架，实现了登录页面和个人中心页面
- **Web**: 使用 HTML/CSS/JavaScript，实现了响应式登录页面

详细代码可参考项目文件：

- iOS: `ContentView.swift` 和 `HomeView.swift`
- Web: `web_login.html`

八、平台差异对比

对比项	Android	iOS	Web
开发语言	Kotlin	Swift	HTML/CSS/JavaScript
UI框架	XML + Material Design	SwiftUI	原生HTML/CSS
页面跳转	Intent	NavigationLink	DOM操作
数据传递	Intent.putExtra	@Binding	JavaScript变量
运行环境	Android设备/模拟器	iOS设备/模拟器	浏览器
构建工具	Gradle	Xcode	无（直接打开）
输入验证	EditText.error	自定义验证	DOM操作
状态管理	Activity生命周期	@State	JavaScript变量

九、遇到的问题及解决方案

问题	解决方案
Gradle下载超时	使用国内镜像源或配置本地Gradle
模拟器无法启动	启用BIOS虚拟化，重启电脑
输入验证提示不显示	使用EditText.error属性显示错误信息
数据传递失败	使用Intent.putExtra()和Intent.getStringExtra()正确传递和接收数据
Web页面布局错乱	使用CSS Flexbox实现响应式布局
iOS页面跳转失败	使用NavigationLink和@State管理状态

十、实验总结

完成情况

-  Android登录页面功能完整
-  iOS登录页面功能完整
-  Web登录页面功能完整
-  Activity跳转与数据传递正常
-  输入验证逻辑正确
-  平台差异对比报告完成

技术收获

1. **Android开发**：掌握了Kotlin语言、Material Design组件、Intent页面跳转和数据传递
2. **iOS开发**：理解了SwiftUI框架、@State状态管理、NavigationLink页面跳转
3. **Web开发**：学会了HTML/CSS/JavaScript、响应式布局、DOM操作
4. **跨平台对比**：了解了不同平台的技术差异和劣势
5. **问题解决**：学会了如何解决Gradle配置、模拟器启动等环境问题

技术实现亮点

1. **统一的登录凭证**：三个平台使用相同的学号和密码
2. **Material Design风格**：Android页面采用现代Material Design
3. **响应式布局**：Web页面适配不同屏幕尺寸
4. **完整的功能**：包含输入验证、页面跳转、退出登录
5. **多平台实现**：覆盖Android、iOS和Web三个平台

后续改进

1. 功能扩展：

- 增加用户注册功能
- 实现忘记密码功能
- 添加网络请求验证
- 集成第三方登录

2. 用户体验优化：

- 增加加载动画
- 优化错误提示
- 添加主题切换功能
- 实现记住密码

3. 技术提升：

- 学习Kotlin Coroutines
- 掌握Jetpack Compose
- 了解Android架构组件
- 学习SwiftUI高级特性
- 掌握前端框架（React/Vue）

十一、附录

项目结构

```
cg2ospdi/
├── app/src/main/
│   ├── java/com/example/loginapp/
│   │   ├── LoginActivity.kt    # Android登录页面
│   │   └── HomeActivity.kt     # Android个人中心
│   ├── res/layout/
│   │   ├── activity_login.xml  # 登录界面
│   │   ├── activity_home.xml   # 个人中心界面
│   │   └── AndroidManifest.xml # 应用清单
│   └── web_login.html          # Web登录页面
├── 2023210634+王智宇+实验一Android登录页面设计.md # 实验一报告
└── 2023210634+王智宇+实验二原生应用开发.md        # 实验二报告
```

测试账号

- 学号：2023210634
- 密码：123456
- 姓名：王智宇

测试结果

平台	登录功能	输入验证	页面跳转	退出登录
Android	✓	✓	✓	✓
iOS	✓	✓	✓	✓
Web	✓	✓	✓	✓

实验报告完成时间：2026年3月21日